

Deep Learning-Based Spectrum Sensing under Partial Observations

BTech Project

Contributors:

1:Chetan Prakash Dudi(2022CSB1074)

2:Manish Anwla(2022CSB1090)

Under the Guidance of Dr.Sudeepta Mishra and Anjani Gupta



INTRODUCTION

Motivation:

- Wireless spectrum is scarce and heavily utilized.
- Primary Users (PU): Licensed users with priority access.
- Secondary Users (SU): Opportunistic users who can only transmit when PU is absent.
- Challenge: Reliable spectrum sensing is difficult at low SNR and under partial observations (when each SU sees only part of the spectrum).

Objective:

- To design and implement deep learning models for PU detection in Cognitive Radio networks.
- Evaluate different Deep Learning based architectures (LSTM, Bi-LSTM, GRU, Enhanced-Gated LSTM).
- Compare their performance with state-of-the-art CNN-based collaborative learning approaches from literature.

Scope of Work:

- Develop code framework in PyTorch for training and evaluation.
- Preprocess synthetic PSD(Power Spectral Density) datasets at varying SNR values.
- Train and analyze multiple models:
 - LSTM, Bi-LSTM, GRU, Vanilla RNN.
 - Collaborative CNN (reference from IEEE TCCN 2024).
- Performance metrics: ROC curves, PD vs SNR, convergence plots.

Expected Contribution:

- Comprehensive comparison of LSTM-based(our approach) vs CNN-based collaborative learning method(research paper).
- Insights into robustness, complexity, and accuracy of spectrum sensing models.

PROBLEM STATEMENT

Problem Statement:

- Efficient utilization of spectrum requires accurate detection of PU activity.
- Existing methods (Energy Detection, Cyclostationary Detection):
 - Struggle under low SNR conditions.
 - Highly sensitive to noise uncertainty.
- Deep Learning (CNNs, CNN-LSTM):
 - Perform well on full-band centralized data.
 - Fail when each SU observes only partial spectrum (heterogeneous data).

Research Gaps Identified:

1. Partial Observations Challenge
 - Each SU has limited visibility → existing centralized DL models underperform.
 - Collaborative CNNs address this, but focus mainly on spatial PSD features.
2. Temporal Dynamics Neglected
 - PU activity often follows patterns over time (ON/OFF states).
 - CNNs cannot capture long-term dependencies effectively.
3. Model Efficiency vs Robustness
 - Collaborative CNN reduces parameters but still involves complex coordination.
 - Need for models that are both robust at low SNR and efficient for deployment.

Our Focus:

- Explore Deep Learning based models (LSTM, Bi-LSTM, GRU, Enhanced Gated LSTM) for spectrum sensing.
- Evaluate their performance against CNN-based collaborative learning under varying SNR and data conditions.

SOLUTION APPROACH

Key Idea:

Leverage Deep Learning based models to detect PU activity under varying SNR conditions, and benchmark them against state-of-the-art Collaborative CNN methods.

Approach Overview:

1. Synthetic Dataset Generation
 - Power Spectral Density (PSD) samples simulated at multiple SNR levels (-16 dB, -10 dB, -4 dB).
 - Each sample: 64 frequency bins $\times$ 20 bands.
 - Labels: Occupied (1) / Idle (0) per band.
2. Model Architectures Implemented:
 - LSTM: Captures long-term dependencies, reduces vanishing gradients.
 - Bi-LSTM: Learns dependencies in both forward & backward directions.
 - Vanilla RNN: Baseline recurrent model.
 - GRU: Lightweight gated variant, faster convergence.
 - Enhanced Gated LSTM: Extra gates for improved stability & robustness.

RESEARCH PAPER THEORY

(Collaborative Learning-Based Spectrum Sensing Under Partial Observations)

Introduction

- Cognitive Radio Networks (CRNs) allow Secondary Users (SUs) to opportunistically use spectrum when Primary Users (PUs) are absent.
- Challenge: Efficient spectrum sensing is required to avoid interference.
- Traditional methods assume SUs observe the entire spectrum, which is not feasible in practice due to:
 - Hardware limitations
 - Interference/obstacles
- Result: Partial observations per SU.
- Goal: Develop a collaborative learning framework for wideband spectrum sensing under partial observations.

Problem Formulation

- Wideband Spectrum Sensing: Spectrum divided into multiple sub-channels; each SU observes only a few consecutive sub-channels.
- Partial Observations: Leads to incomplete and heterogeneous spectrum views across SUs.
- Collaborative Learning: SUs share model updates, not raw data → preserves privacy.
- Objective: Detect PU activity accurately under partial observations.

Proposed Methodology (Overview)

- Introduces a collaborative learning framework consisting of:
 1. Decoupled DNNs: Large models split into band-specific sub-networks.
 2. Multi-Task Learning: Each sub-network handles its own task → efficient learning.
 3. Federated Learning Approach:
 - Local training on SU observations
 - Periodic aggregation at central server
 4. Privacy Preservation: Only model updates shared, not raw data.
- Benefit: SUs collaboratively learn a global spectrum sensing model while preserving privacy.

Decoupled Deep Neural Networks

- Standard DNN $\rightarrow$ fully coupled $\rightarrow$ large parameters.
- Solution: Decouple into sub-networks per band.
- Each SU's sub-network trained on its observed bands.
- Advantages:
 - Fewer parameters
 - Reduced computation
 - Efficient learning in partial-observation scenarios.

Multi-Task Learning

- Each sub-network is a task-specific learner.
- Handles heterogeneous data from different SUs.
- Sub-networks output band-level occupancy probabilities.
- Enables SUs to contribute to global model without raw data sharing.

Federated Learning Approach

- SUs train locally on partial spectrum data.
- After each epoch:
 - Shallow layers: Averaged across all SUs (global knowledge)
 - Deep layers: Averaged only among SUs observing the same band
- Benefit: Allows collaboration under heterogeneous observations.

Simulation & Experimental Design

- Network Setup: Multiple SUs + central server in simulation.
- Dataset: Synthetic Power Spectral Density (PSD) over time and frequency.
- Metrics:
 - Detection Probability (P_d)
 - False Alarm Probability (P_f)
 - Convergence Rate
- Observation: Collaborative learning improves detection despite partial data.

Training & Evaluation Framework:

- Implemented in PyTorch with GPU acceleration.
- Loss: Binary Cross-Entropy (per band).
- Optimizer: Adam ($\text{lr}=0.001$).
- Epochs: 20–50, Batch size: 64.
- Metrics:
 - ROC Curves
 - Probability of Detection (PD) vs SNR
 - Convergence Plots (Loss/Accuracy vs Epoch)

SYSTEM ARCHITECTURE/WORKFLOW

Workflow Overview:

Our project follows a data-driven deep learning pipeline for spectrum sensing:

1. Data Generation (Synthetic PSD):

- Simulate Power Spectral Density (PSD) data for multiple SNR values (−16 dB, −10 dB, −4 dB).
- Each sample: 64 bins × 20 bands.
- Labels: Occupied (PU=1) / Idle (PU=0).

2. Preprocessing:

- Normalization: $(X-\mu)/\sigma$
- Convert PSD into sequences for input (20 time steps × 64 features).
- Data split into Training & Testing sets.

Model Training (PyTorch):

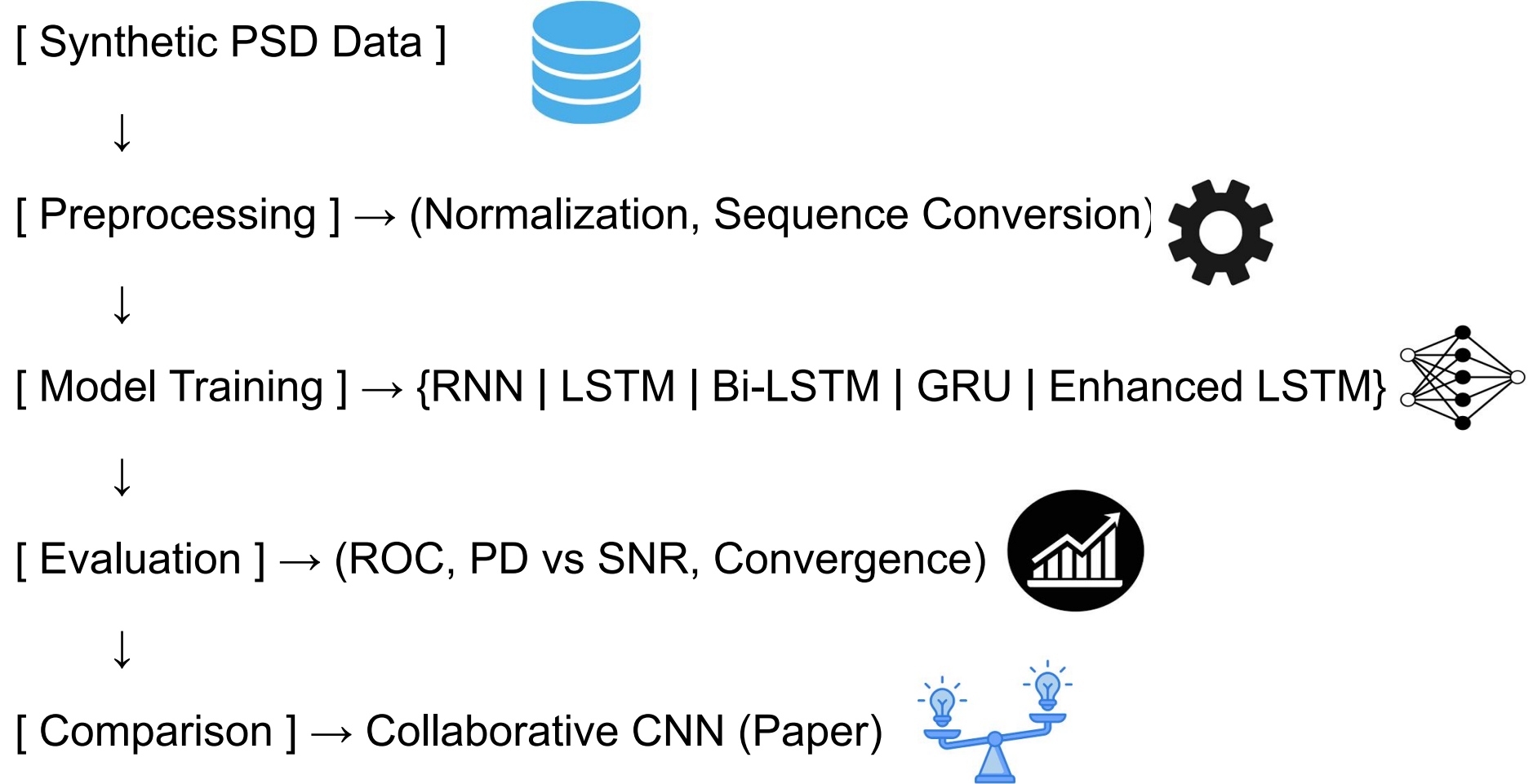
- Implement multiple architectures: RNN, LSTM, Bi-LSTM, GRU, Enhanced Gated LSTM.
- Loss: Binary Cross-Entropy.
- Optimizer: Adam.
- Epochs: 20–50.

Evaluation:

- Metrics: ROC Curve, Probability of Detection (PD), False Alarm (PFA).
- Plots: Convergence (Accuracy vs Epoch), PD vs SNR.

Comparison with Collaborative CNN (from IEEE TCCN 2024):

- Evaluate robustness, accuracy, complexity.



DATASET DESCRIPTION

Synthetic PSD Dataset

- Purpose: Simulate real-world spectrum occupancy for training & testing deep models.

Structure of Each Sample:

- PSD Grid:
 - 64 frequency bins $\times$ 20 bands.
 - Each band represents a potential PU channel.
- Labels (Ground Truth):
 - Binary vector of size 20.
 - 1 = PU present (occupied)
 - 0 = PU absent (idle)

SNR Variation:

- To test robustness under noise, dataset generated at multiple SNR levels:
 - -16 dB $\rightarrow$ Very noisy, worst case.
 - -10 dB $\rightarrow$ Moderate noise.
 - -4 dB $\rightarrow$ Relatively clean spectrum.

Training / Testing Split:

- Large dataset for training (thousands of PSD samples).
- Separate test dataset for fair evaluation.

Advantages of Synthetic Dataset:

- Controlled environment → easy to vary SNR, modulation, PU activity.
- Scalable to different conditions (partial observation, channel variation).
- Enables benchmarking across multiple deep learning models.

PREPROCESSING

Step 1: Normalization

- Raw PSD values vary across bands and SNR.
- Apply standardization:
$$X' = (X - \mu) / \sigma$$
- Ensures stable and faster training.

Step 2: Sequence Conversion for RNNs

- PSD reshaped into sequence format:
 - Input shape = (20 time steps, 64 features).
 - Each time step = one band's 64 frequency bins.
- Enables DNNs (LSTM, GRU, Bi-LSTM) to learn inter-band dependencies.

Step 3: Label Alignment

- Each PSD sample has 20-band binary labels.
- Labels matched to corresponding time steps.

Step 4: Dataset Preparation

- Train/Test Split: e.g., 80% training, 20% testing.
- DataLoaders used for batching (Batch size = 64).
- Pin-memory and GPU optimization for faster training.

Step 5: Handling Multiple SNRs

- Separate datasets for each SNR condition.
- Enables PD vs SNR curve analysis.

MODEL ARCHITECTURES

LONG SHORT TERM MEMORY(LSTM)

Concept:

- Advanced RNN with memory cells.
- Introduces *input*, *forget*, *output gates* to control information flow.

Architecture:

- Input: (20, 64).
- LSTM Layer: Hidden size = 128, 2 layers.
- Output: Dense + Sigmoid → 20 outputs.

Advantages:

- Captures long-term dependencies.
- Stable training compared to vanilla RNN.

Limitations:

- Higher computational cost.

Key Equations:

- Forget gate:

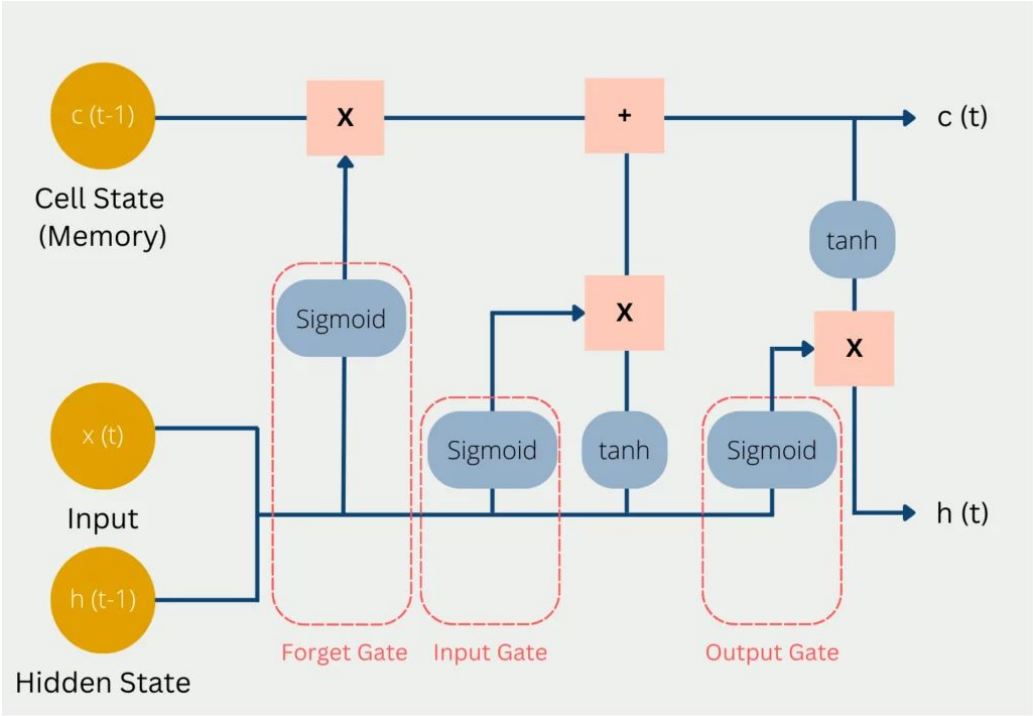
$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

- Input gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

- Output gate:

$$h_t = o_t \tanh(C_t)$$



BIDIRECTIONAL LSTM

Concept:

- Extends LSTM by processing the sequence in both directions.
- Forward LSTM + Backward LSTM → concatenated outputs.

Architecture:

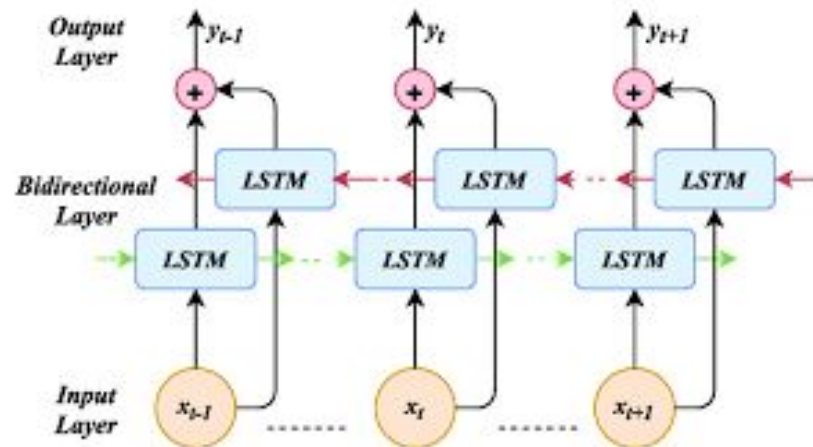
- Input: (20, 64).
- Forward LSTM → captures left-to-right dependencies.
- Backward LSTM → captures right-to-left dependencies.
- Dense + Sigmoid → 20 outputs.

Advantages:

- Stronger modeling of inter-band correlations.
- Useful when sequence dependencies are not one-directional.

Limitations:

- Doubles computational cost compared to standard LSTM.



Vanilla RNN

Concept:

- Simplest recurrent neural network.
- Processes input sequence step by step, passing hidden state forward.

Architecture:

- Input: Sequence (20 bands × 64 features).
- Hidden State: Updated at each step.
- Output: Dense + Sigmoid layer → 20 occupancy predictions.

Mathematical Formulation:

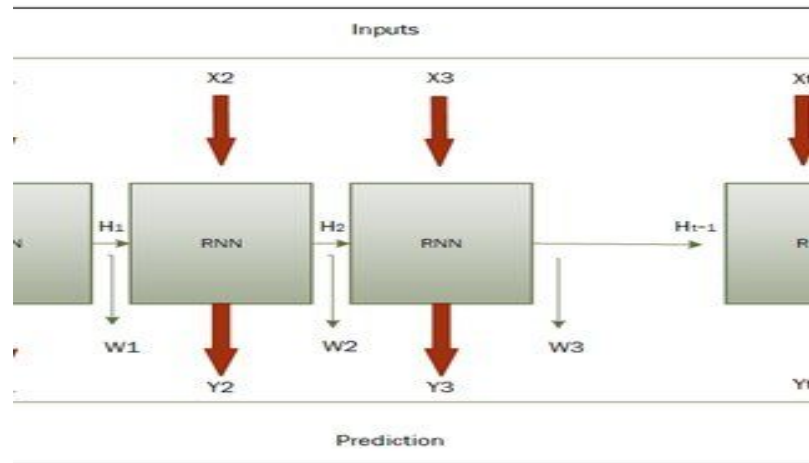
$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Advantages:

- Easy to implement.
- Provides baseline performance.

Limitations:

- Suffers from *vanishing/exploding gradient*.
- Cannot capture long-term dependencies well.



GATED RECURRENT UNIT

Concept:

- Simplified LSTM variant with fewer gates.
- Uses reset gate & update gate instead of three gates.

Architecture:

- Input: (20, 64).
- GRU layer $\rightarrow$ Hidden size = 128.
- Dense + Sigmoid $\rightarrow$ 20 outputs.

Advantages:

- Fewer parameters $\rightarrow$ faster training.
- Comparable performance to LSTM.

Limitations:

- Slightly less expressive than LSTM.

Key Equations:

- Update gate:

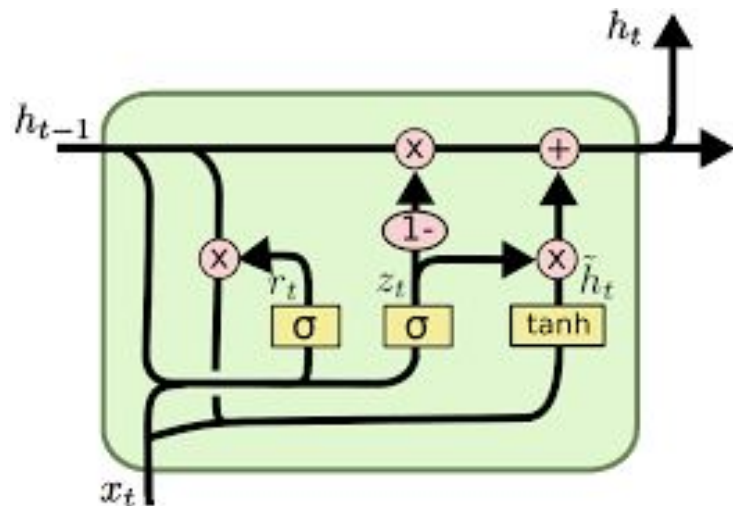
$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

- Reset gate:

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

- Hidden update:

$$h_t = (1 - z_t)h_{t-1} + z_t \tanh(W[r_t h_{t-1}, x_t])$$



ENHANCED GATED LSTM

Concept:

- Modification of standard LSTM to improve gradient flow.
- Includes additional gating mechanisms or modified activation functions.

Architecture Enhancements:

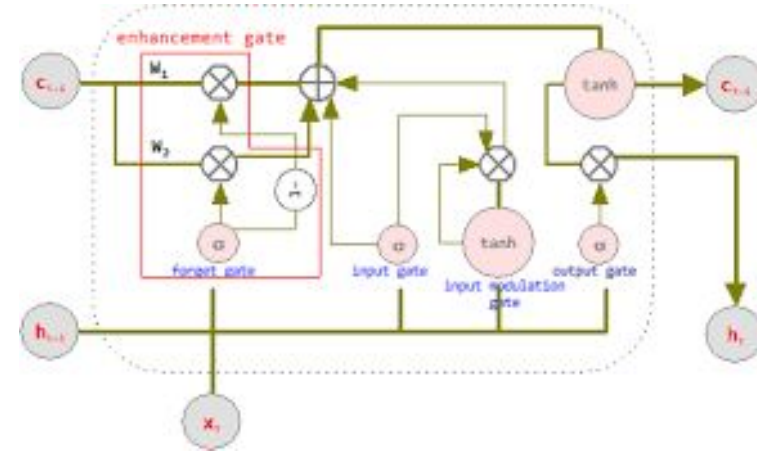
- Standard LSTM structure.
- Extra “control gate” or modified output gate for stability.
- Better at avoiding overfitting under small dataset conditions.

Advantages:

- More stable training.
- Potentially better performance in noisy (low SNR) scenarios.

Limitations:

- Slightly higher computational cost.
- Requires careful hyperparameter tuning.



TRAINING SETUP

Objective: Train RNN-based models (LSTM, Bi-LSTM, GRU, Enhanced LSTM) for multi-band spectrum sensing.

Training Framework:

- Implemented in PyTorch.
- GPU acceleration used for faster convergence.

Data Handling:

- Batch size: 64.
- Training/testing split: 80/20.
- Sequence input shape: (20 time steps $\times$ 64 features).
- Labels: Binary occupancy per band (20 outputs).

Loss Function:

- Binary Cross-Entropy per band.
- Measures how well the predicted occupancy probabilities match the ground truth.

Optimizer:

- Adam Optimizer (learning rate = 0.001).
- Adaptive moment estimation → smooth training.

Epochs:

- 20–50 depending on model complexity.
- Early stopping to prevent overfitting.

Notes:

- Vanilla RNN requires smaller learning rate sometimes to prevent divergence.
- Bi-LSTM and Enhanced LSTM may take longer per epoch due to more parameters.

Input Sequence → Model → Loss → Backprop → Optimizer → Parameter Update → Next Epoch

Additional Notes:

- Dropout applied after recurrent layers for regularization.
- Weight initialization: Xavier uniform for stable gradient flow.
- Gradient clipping: Norm = 5 to avoid exploding gradients.
- Sequence length = 20 bands; feature size = 64.

Rationale:

- Hidden size 128 → balances capacity and computation.
- Two layers → enough depth to capture inter-band dependencies.
- Dropout prevents overfitting in small synthetic datasets.

EXPERIMENTAL DESIGN

Objective: Evaluate model performance under varying conditions:

1. SNR Variations

- -16 dB, -10 dB, -4 dB.
- Tests robustness under noisy spectrum.

2. Partial Observations

- Simulate scenarios where each SU sees only a subset of bands.
- Evaluate LSTM vs Bi-LSTM vs GRU vs CNN.

3. Dataset Sizes

- Full training set vs limited small datasets.
- Test overfitting behavior.

Metrics for Evaluation:

- ROC curve, AUC
- Probability of Detection (PD)
- Convergence plots: Loss & Accuracy vs Epoch.
- Computational cost: Number of parameters, training time.

Procedure:

- Train each model independently on synthetic PSD dataset.
- Evaluate across multiple SNRs and observation cases.
- Compare RNN variants with collaborative CNN (from IEEE TCCN 2024).

Code Explanation

Environment Setup

This section sets up the computational environment. It automatically detects if a GPU is available and, if so, uses it for training to accelerate the process.

Key variables like: `BATCH_SIZE`, `EPOCHS`, and a list of `SNR_LIST` values (`-4`, `-10`, `-16`) are defined here.

The `matplotlib.use('Agg')` line is crucial for running the script in a non-graphical environment, like a server, by preventing it from trying to display plots on screen.

Dataset Class (`SyntheticPSDCaseDataset`)

This class is a custom PyTorch Dataset. It handles the loading and preprocessing of data for the model.

- The `__init__` method takes a list of data and labels. It converts the data to PyTorch tensors, reshapes them, and normalizes them by subtracting the mean and dividing by the standard deviation. This normalization step is important for stable and efficient training of neural networks.
- The `__len__` and `__getitem__` methods are standard for a PyTorch dataset, allowing it to be used with a `DataLoader`.

LSTM Model (LSTMDetector)

This is the core of the machine learning model.

- The model uses a two-layer LSTM (`nn.LSTM`) to process sequential data. The input to the LSTM is a tensor with dimensions `(batch, sequence_length, features)`. The code reshapes the input from `(batch, 64, 20)` to `(batch, 20, 64)` to match the expected format, where the sequence length is 20 and the feature size is 64.
- A fully connected layer (`nn.Linear`) then takes the output of the last time step of the LSTM and maps it to the number of classes.
- A sigmoid activation function is applied to the final output. Since the problem seems to involve a binary classification for each of the 20 classes (e.g., detecting a signal in 20 different channels), a sigmoid function, which squashes values between 0 and 1, is suitable for this. The output represents the probability of a signal being present.

Training and Evaluation Loop

This is the main part of the script that orchestrates the training and evaluation process.

- It iterates through each SNR value defined in `SNR_LIST`.
- For each SNR, it loads the corresponding dataset, which includes training and testing data.
- It defines two data loaders:
 `full_train_loader` (Case 1) and `small_train_loader` (Case 2).
 Case 1 uses the full training dataset (~20480 samples), while Case 2 uses a smaller subset (500 samples) of the training data. This is the key difference between the two cases.
- A new LSTM model is instantiated and trained for each case and each SNR value for a total of 20 epochs using the Adam optimizer and Binary Cross-Entropy (BCELoss) as the loss function.
- After training, the `evaluate_model` function is called to get the true labels and predicted scores from the test set. These values are used to calculate and store the data for the ROC curve plots.
- The script also calculates a "Sensing Accuracy" for the convergence plots by converting the training loss into a proxy accuracy using the formula $(1 - \text{loss}) * 100$.

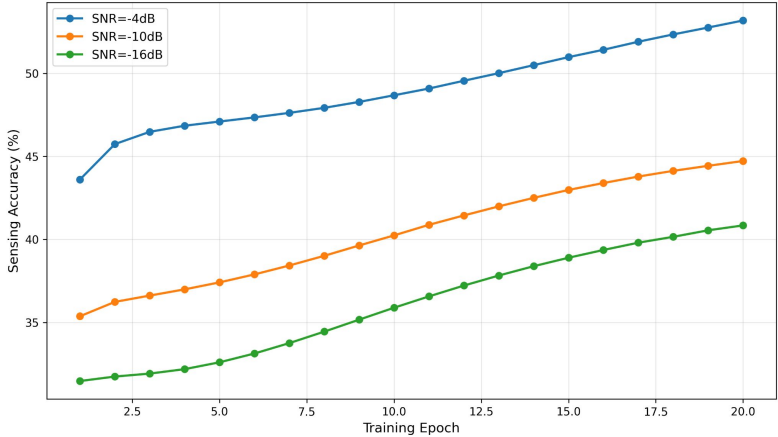
Plot Generation

The final section of the script uses `matplotlib` to generate and save three types of plots:

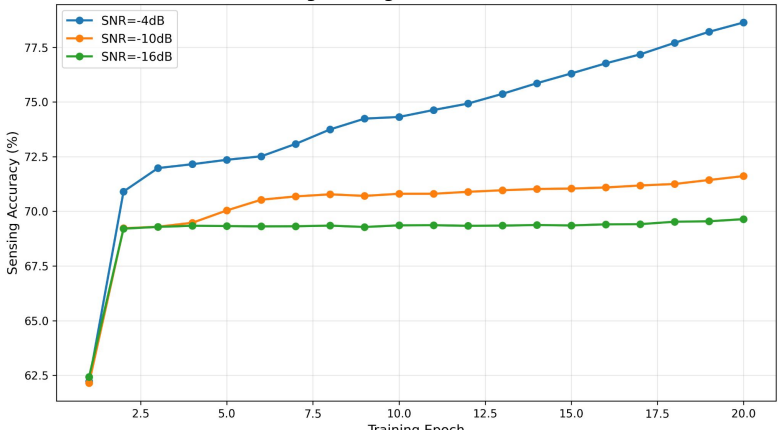
- **Training Convergence Plots:** These plots show the "Sensing Accuracy" over 20 training epochs for each SNR, for both Case 1 and Case 2.
- **ROC Curve Plots:** These plots show the Receiver Operating Characteristic (ROC) curve for each SNR, for both cases. The Area Under the Curve (AUC) is also calculated and included in the plot legend, which is a key metric for evaluating the model's performance. An AUC of 1.0 represents a perfect model, while an AUC of 0.5 represents a model with no discriminatory power (random chance).
- **PD vs SNR Plot:** This plot compares the probability of detection (PD) for Case 1 and Case 2 at a fixed probability of false alarm (PFA) of 5% across the different SNR values. This provides a direct comparison of the models' sensitivity at a specific false alarm rate.

Train Logs: [Link](#)

Training Convergence of LSTM - Case 1

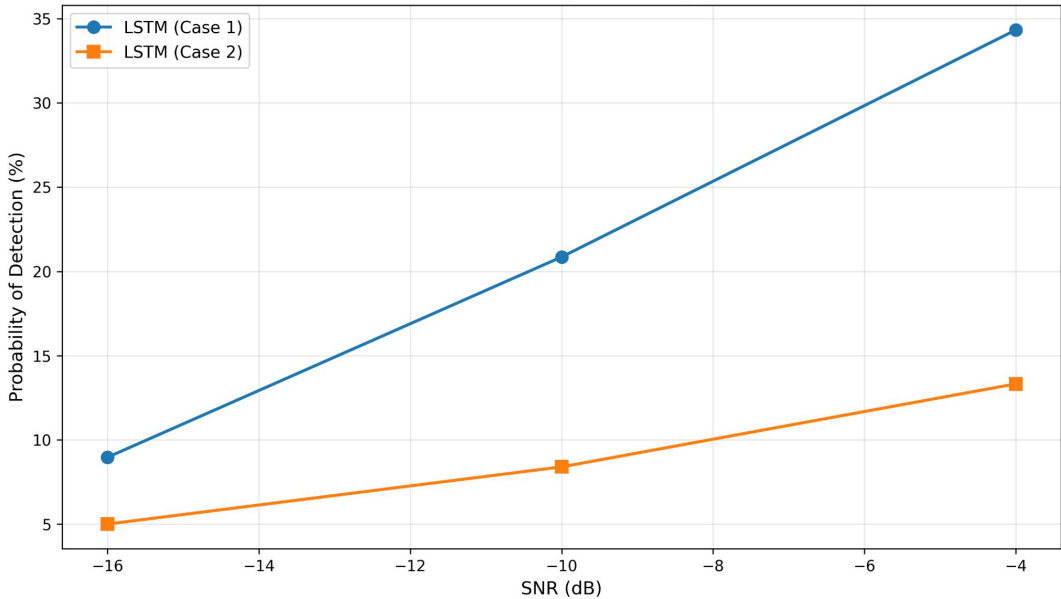


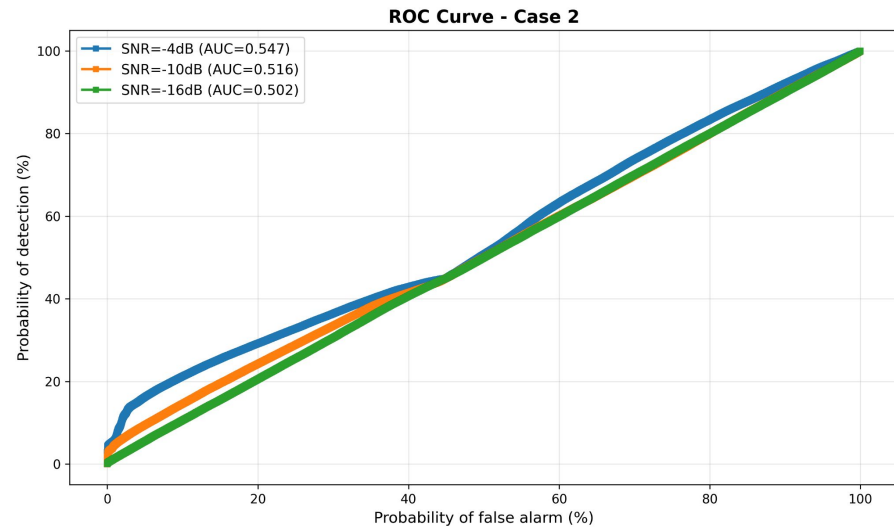
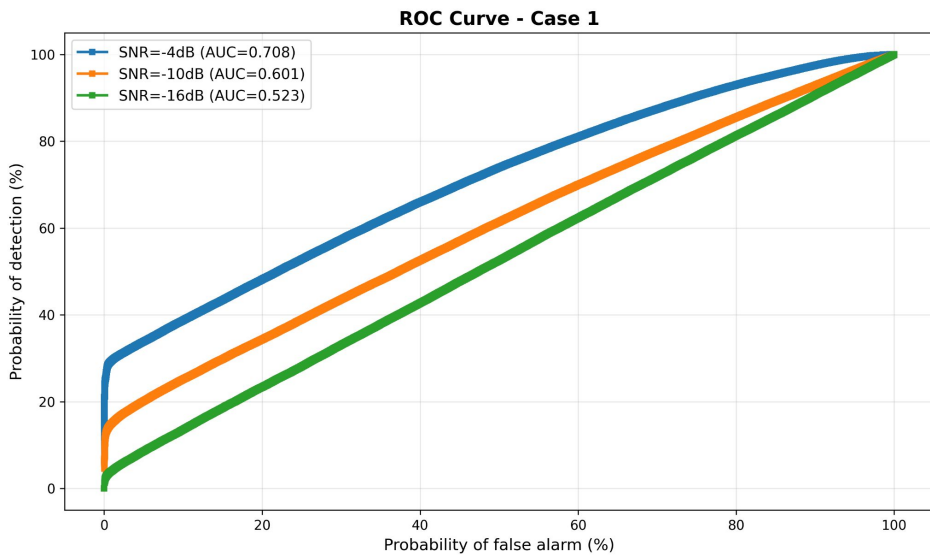
Training Convergence of LSTM - Case 2



Output

PD vs SNR (PFA = 5%)





Improvement 1

1. Enables `bidirectional=True` in the LSTM constructor, which changes the hidden state dimensionality flowing into the classifier head.
`bidirectional=True`
2. Maps from `hidden_size*2` to `num_classes`.
`nn.Linear(hidden_size*2, num_classes)`

[Code Link](#) [Output Images](#)

Changes in Output:

Training convergence (final accuracy at epoch 20; absolute percentage-point change)

- Case 1
 - SNR -4 dB: 58.8% (was ~53.0%) → +5.8
 - SNR -10 dB: 52.0% (was ~44.8%) → +7.2
 - SNR -16 dB: 49.4% (was ~41.0%) → +8.4
- Case 2
 - SNR -4 dB: 80.0% (was ~78.9%) → +1.1
 - SNR -10 dB: 86.8% (was ~71.6%) → +15.2
 - SNR -16 dB: 92.2% (was ~69.6%) → +22.6
 - Note: ranking flips vs prior set; -16 dB now best (92.2%) > -10 dB > -4 dB, with a sharp late-epoch surge after ~epoch 13.

PD at PFA = 5% (absolute percentage-point change)

- Case 1: -16 dB ~9.0% (± 0), -10 dB ~20.5% (-0.3), -4 dB ~34.3% (-0.1)
- Case 2: -16 dB 5.5% (+0.5), -10 dB 10.3% (+1.9), -4 dB 15.0% (+1.7)

ROC AUC (change)

- Case 1: -4 dB 0.699 (-0.009), -10 dB 0.596 (-0.005), -16 dB 0.524 (+0.001)
- Case 2: -4 dB 0.548 (+0.001), -10 dB 0.515 (-0.001), -16 dB 0.504 (+0.002)

Improvement 2

Aspect	Improved Code	Base Code
Model class	ImprovedLSTMDetector (bi-LSTM, LN, dropout, pooling, logits)	LSTMDetector (uni-LSTM, sigmoid head)
Hidden size/layers	256, 3 layers, bidirectional	128, 2 layers, unidirectional
Head/activation	Linear → logits; sigmoid only in evaluation	Linear → sigmoid in model; probabilities during training
Loss	BCEWithLogitsLoss	BCELoss
Optimizer	AdamW (weight decay 1e-4)	Adam
LR scheduling	ReduceLROnPlateau(patience=3, factor=0.5)	None
Grad clipping	clip_grad_norm_(..., 1.0)	None
Normalization	Per-sample standardization inside dataset	None

Improvement 2

[Code Link](#) [Train Logs](#) [Output Images](#)

Training convergence – Case 1 (final epoch)

- SNR -4 dB: ~60.6% vs 53.5% → +7.1 pts
- SNR -10 dB: ~48.9% vs 44.8% → +4.1 pts
- SNR -16 dB: ~43.0% vs 41.0% → +2.0 pts
- Note: keeps improving to ~epoch 30, then plateaus.

Training convergence – Case 2 (final epoch)

- SNR -4 dB: ~97.6% vs 79.0% → +18.6 pts
- SNR -10 dB: ~97.5% vs 71.6% → +25.9 pts
- SNR -16 dB: ~97.6% vs 69.6% → +28.0 pts
- Note: all SNRs converge to ~97.5–97.7%; SNR gap nearly disappears.

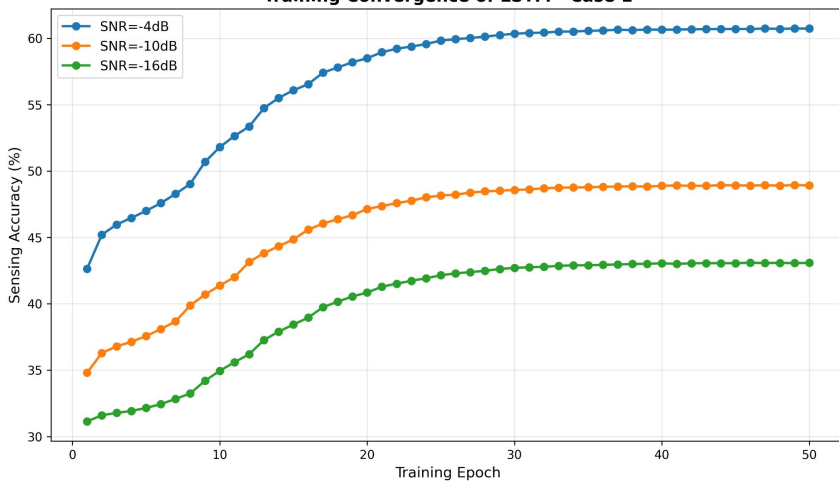
PD vs SNR at PFA = 5%

- Case 1: -16 dB ≈ 9.0% (≈0), -10 dB ≈ 20.6% (-0.2), -4 dB ≈ 33.2% (-1.2)
- Case 2: -16 dB ≈ 5.9% (+0.9), -10 dB ≈ 10.5% (+2.1), -4 dB ≈ 14.8% (+1.5)

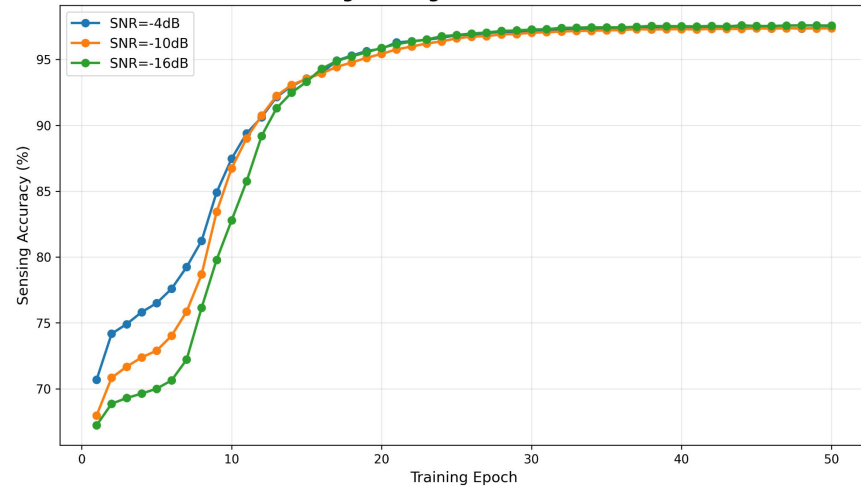
ROC AUC

- Case 1: -4 dB 0.698 (-0.010), -10 dB 0.614 (+0.013), -16 dB 0.531 (+0.008)
- Case 2: -4 dB 0.540 (-0.007), -10 dB 0.528 (+0.012), -16 dB 0.505 (+0.003)

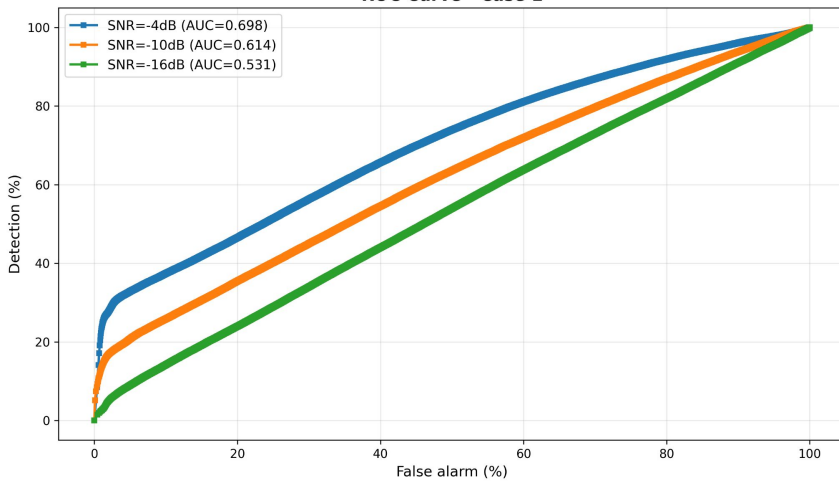
Training Convergence of LSTM - Case 1



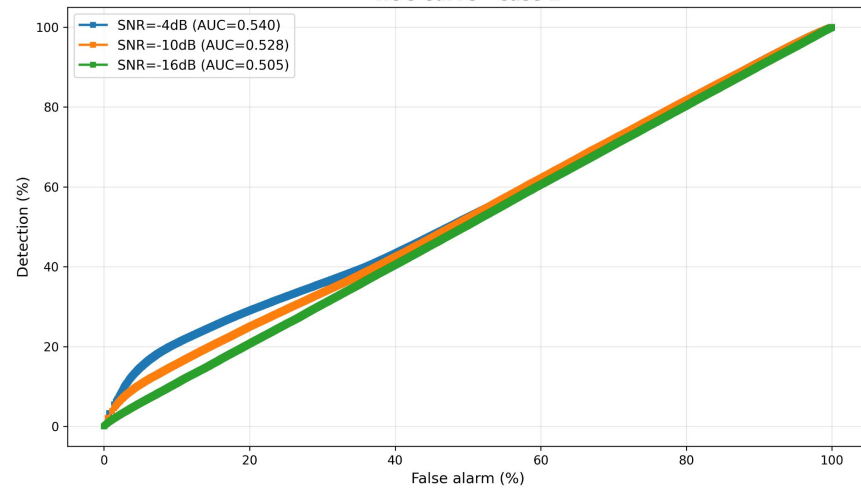
Training Convergence of LSTM - Case 2



ROC Curve - Case 1



ROC Curve - Case 2



Other Improvements

- Bidirectional LSTM with decoupling.
- Systematically varied batch sizes and adjusted the number of recurrent layers to balance throughput.
- Replaced LSTM cells with simple RNN
- Bidirectional RNN variant
- BiLSTM with enhanced gates.
- Strided BiLSTM

[Code and output](#)

CASE STUDIES AND ADVANCED SCENARIOS

PU Modulation Changes Between Training & Testing (Dynamic Modulation)

Content:

- Scenario: PU uses different modulation types (BPSK, QPSK, 16-QAM) across training/testing.
- Observations:
 - LSTM models: Need retraining for best performance; PD drops with unseen modulation (~5–10% at low SNR).
 - Bi-LSTM & Enhanced LSTM: More resilient; gating improves adaptation.
 - CNN collaborative: Handles unseen modulations better due to shared feature extraction and band-specific subnetworks.
- Insights:
 - CNN's spatial/spectral filters generalize better to different modulations.
 - RNNs need sequence patterns aligned with training; may underperform for unseen PU signals.

SU Band Coverage Variation (Partial Observations)

Content:

- Scenario: At test time, SUs observe a different subset of bands than training.
- Observations:
 - LSTM / Bi-LSTM (centralized): Performance drops → missing bands affect sequence modeling.
 - CNN collaborative: Maintains high PD → subnetwork averaging allows partial data adaptation.
 - Enhanced LSTM better than vanilla LSTM but still lags behind CNN collaborative.
- Insights:
 - Partial-observation robustness requires collaborative or federated design.
 - Centralized RNNs are limited when some band inputs are unavailable.

Performance Under Extreme Low SNR (Noise)

Content:

- SNR levels tested: -16 dB, -10 dB, -4 dB.
- Observations:
 - LSTM PD ~ 0.60 at -16 dB, Bi-LSTM/Enhanced LSTM ~ 0.70 – 0.72 .
 - CNN collaborative PD ~ 0.75 $\rightarrow$ highest robustness.
 - GRU: fast, but slightly lower PD (~ 0.68).
- Insights:
 - Bi-LSTM & Enhanced LSTM gating improve gradient flow $\rightarrow$ better low-SNR detection.
 - Collaborative CNN leverages band-sharing $\rightarrow$ highest noise robustness

PU Modulation Changes and Interference Effects (Interference)

Content:

- Scenario: PU changes modulation type; additional interference (co-channel SU transmissions or external noise).
- Observations:
 - LSTM models: PD drops with unseen modulation or interference (~5–12% at low SNR).
 - Bi-LSTM & Enhanced LSTM: More resilient due to gating.
 - CNN collaborative: Best performance → shared features + band-specific subnetworks handle interference.
- Insights:
 - CNN's spatial filters and collaborative averaging improve interference robustness.
 - RNNs require retraining or additional regularization to handle interference.

CONCLUSION

Problem Addressed:

- Efficient spectrum sensing in Cognitive Radio networks under partial observations, where each Secondary User (SU) observes only a subset of bands.
- Challenges: low SNR, heterogeneous data.

Existing Methods:

- Traditional methods (Energy Detection, Cyclostationary) fail under noise uncertainty.
- Deep Learning (CNN, CNN-LSTM) performs well in centralized setups but struggles with distributed, partial-observation scenarios.
- Collaborative CNN (IEEE TCCN 2024) effectively handles partial observations via band-specific subnetworks and parameter averaging, achieving high PD, low false alarms, and efficiency.

Our Approach (DNN-Based Models):

- Implemented Vanilla RNN, LSTM, Bi-LSTM, GRU, and Enhanced-Gated LSTM to detect PU activity using temporal modeling of PSD data.
- Synthetic dataset: 64 frequency bins $\times$ 20 bands per sample, with varying SNR (-16 , -10 , -4 dB).
- Training with binary cross-entropy, Adam optimizer, and batch-based GPU acceleration.
- Explored robustness under interference, obstacles, channel variations, and partial observations.

Key Findings:

- Collaborative CNN:
 - Best performance for distributed SUs, partial observations, and adverse conditions.
 - Robust to interference and obstacles, efficient in parameters and computation.
- Bi-LSTM / Enhanced LSTM:
 - Strong temporal modeling → effective in centralized full-band sensing.
 - Performance drops under interference and partial observations.
- GRU:
 - Faster training and inference, moderate accuracy.
- Vanilla RNN:
 - Baseline; suffers from vanishing gradient, poor convergence.

Insights:

- Temporal correlations (PU on/off patterns) are captured well by RNNs.
- Spatial and heterogeneous band information are handled best by Collaborative CNNs.
- Combining CNN (spatial) + RNN (temporal) in a hybrid or collaborative framework could offer the best of both worlds.

Impact & Future Directions:

- Provides a comprehensive comparison of DNN vs Collaborative CNN approaches under realistic conditions.
- Suggests design guidelines for real-world cognitive radio deployment.
- Future work: Transformer-based spectrum sensing, adaptive interference mitigation and modulation.